RESEARCH PAPER

RIGOR

Retrieval-Augmented Generation of Ontologies from Relational Databases

Nadeen Fathallah[1] · Mojtaba Nayyeri[1] · Athish A Yogi[1]

Ratan Bahadur Thapa[1] · Hans-Michael Tautenhahn[3]

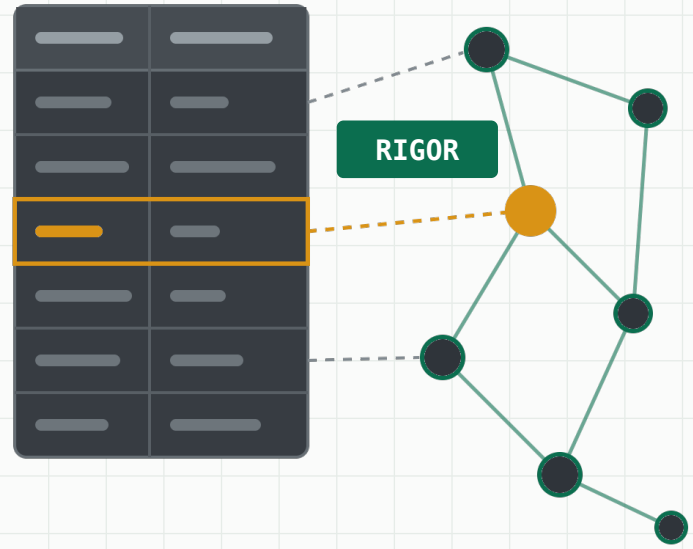
Anton Schnurpel[3] · Steffen Staab[1,2]

[1] Institute for Artificial Intelligence, University of Stuttgart, Germany

[2] University of Southampton, United Kingdom

[3] Universitätsklinikum Leipzig, Germany

PAPER arXiv:2506.01232 CODE github.com/NadeenAhmad/RIGOR

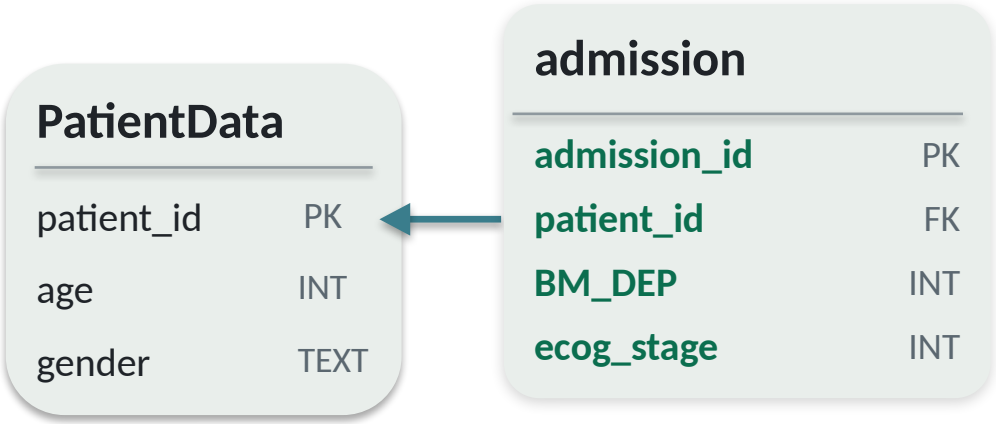


ISWC 2026

Bari, Italy

A database can tell us how data is stored. Can it tell us what the data means?

RELATIONAL SCHEMA · EXCERPT



The FK links an **admission** to a **patient**.

WHAT WE STILL NEED TO UNDERSTAND

01 `patient_id` — How do we represent the **link** from an **admission** to its **patient**?

02 `BM_DEP` — Which **clinical concept** does this **abbreviation mean**?

03 `ecog_stage` — What do the **encoded** score **values mean**?

Those meanings matter for ontology-based access, knowledge graphs and integration.

WHAT WE WANT TO BUILD

Not just a cleaner schema.
A usable semantic model.

INPUT

admission

admission_id	PK
patient_id	FK
BM_DEP	INT
ecog_stage	INT



TARGET

WHAT WE WANT TO BUILD

Not just a cleaner schema. A usable semantic model.

INPUT

admission

admission_id	PK
patient_id	FK
BM_DEP	INT
ecog_stage	INT



TARGET

Admission —hasPatientData→ PatientData

```
Class: Admission
  SubClassOf:
    admission_id some xsd:integer,
    hasPatientData some PatientData,        ← schema (foreign key)
    BM_DEP only xsd:integer,
    ecog_stage only xsd:integer

DataProperty: admission_id
ObjectProperty: hasPatientData              ← schema (foreign key)
DataProperty: BM_DEP
  Annotations:


    prov:wasDerivedFrom admission:BM_DEP    ← source column
DataProperty: ecog_stage
  Annotations:


    prov:wasDerivedFrom admission:ecog_stage ← source column
```

WHAT WE WANT TO BUILD

Not just a cleaner schema. A usable semantic model.

INPUT

admission

admission_id	PK
patient_id	FK
BM_DEP	INT
ecog_stage	INT



TARGET

Admission —hasPatientData→ PatientData

```
Class: Admission
  SubClassOf:
    admission_id some xsd:integer,
    hasPatientData some PatientData,          ← schema (foreign key)
    BM_DEP only xsd:integer,
    ecog_stage only xsd:integer

DataProperty: admission_id
ObjectProperty: hasPatientData                ← schema (foreign key)
DataProperty: BM_DEP
  Annotations:
    rdfs:label "Bone Marrow Depression"@en,   ← documentation

    prov:wasDerivedFrom admission:BM_DEP      ← source column
DataProperty: ecog_stage
  Annotations:
    rdfs:label "ECOG Performance Status"@en,  ← documentation

    prov:wasDerivedFrom admission:ecog_stage  ← source column
```

WHAT WE WANT TO BUILD

Not just a cleaner schema. A usable semantic model.

INPUT

admission

admission_id	PK
patient_id	FK
BM_DEP	INT
ecog_stage	INT



TARGET

Admission —hasPatientData→ PatientData

```
Class: Admission
  SubClassOf:
    admission_id some xsd:integer,
    hasPatientData some PatientData,          ← schema (foreign key)
    BM_DEP only xsd:integer,
    ecog_stage only xsd:integer

DataProperty: admission_id
ObjectProperty: hasPatientData                ← schema (foreign key)
DataProperty: BM_DEP
  Annotations:
    rdfs:label "Bone Marrow Depression"@en,   ← documentation
    skos:relatedMatch obo:DOID_4961,          ← Disease Ontology
    prov:wasDerivedFrom admission:BM_DEP      ← source column
DataProperty: ecog_stage
  Annotations:
    rdfs:label "ECOG Performance Status"@en,  ← documentation

    prov:wasDerivedFrom admission:ecog_stage  ← source column
```

WHAT WE WANT TO BUILD

Not just a cleaner schema. A usable semantic model.

INPUT

admission

admission_id	PK
patient_id	FK
BM_DEP	INT
ecog_stage	INT



TARGET

Admission —hasPatientData→ PatientData

```
Class: Admission
  SubClassOf:
    admission_id some xsd:integer,
    hasPatientData some PatientData,          ← schema (foreign key)
    BM_DEP only xsd:integer,
    ecog_stage only xsd:integer

DataProperty: admission_id
ObjectProperty: hasPatientData                ← schema (foreign key)
DataProperty: BM_DEP
  Annotations:
    rdfs:label "Bone Marrow Depression"@en,   ← documentation
    skos:relatedMatch obo:DOID_4961,          ← Disease Ontology
    prov:wasDerivedFrom admission:BM_DEP      ← source column
DataProperty: ecog_stage
  Annotations:
    rdfs:label "ECOG Performance Status"@en,  ← documentation
    rdfs:comment "0 = fully active … 5 = dead"@en,
    prov:wasDerivedFrom admission:ecog_stage  ← source column
```

WHAT WE WANT TO BUILD

Not just a cleaner schema. A usable semantic model.

INPUT

admission

admission_id	PK
patient_id	FK
BM_DEP	INT
ecog_stage	INT

WHAT THIS ENABLES

Ontology-based data access (OBDA)
Knowledge graph population
Semantic data integration

TARGET

Admission —hasPatientData→ PatientData

```
Class: Admission
  SubClassOf:
    admission_id some xsd:integer,
    hasPatientData some PatientData,        ← schema (foreign key)
    BM_DEP only xsd:integer,
    ecog_stage only xsd:integer

DataProperty: admission_id
ObjectProperty: hasPatientData              ← schema (foreign key)
DataProperty: BM_DEP
  Annotations:
    rdfs:label "Bone Marrow Depression"@en,        ← documentation
    skos:relatedMatch obo:DOID_4961,               ← Disease Ontology
    prov:wasDerivedFrom admission:BM_DEP           ← source column
DataProperty: ecog_stage
  Annotations:
    rdfs:label "ECOG Performance Status"@en,       ← documentation
    rdfs:comment "0 = fully active … 5 = dead"@en,
    prov:wasDerivedFrom admission:ecog_stage       ← source column
```

THE RESEARCH GAP

Two partial solutions. Neither solves the whole problem.

SCHEMA-FIRST RULES

Preserve the structure

Tables, columns and foreign keys can be mapped systematically.

But implicit **domain semantics** remain **missing**.

W3C Direct Mapping [Arenas et al. 2012; Sequeda et al. 2012] · **BootOX** [Jiménez-Ruiz et al., ISWC 2015] · **Ben Mahria et al.** [J. Big Data 2021]

LLM-FIRST GENERATION

Suggest the meaning

Labels, concepts and plausible relationships can be proposed.

But elements may be **omitted** or modeled incorrectly.

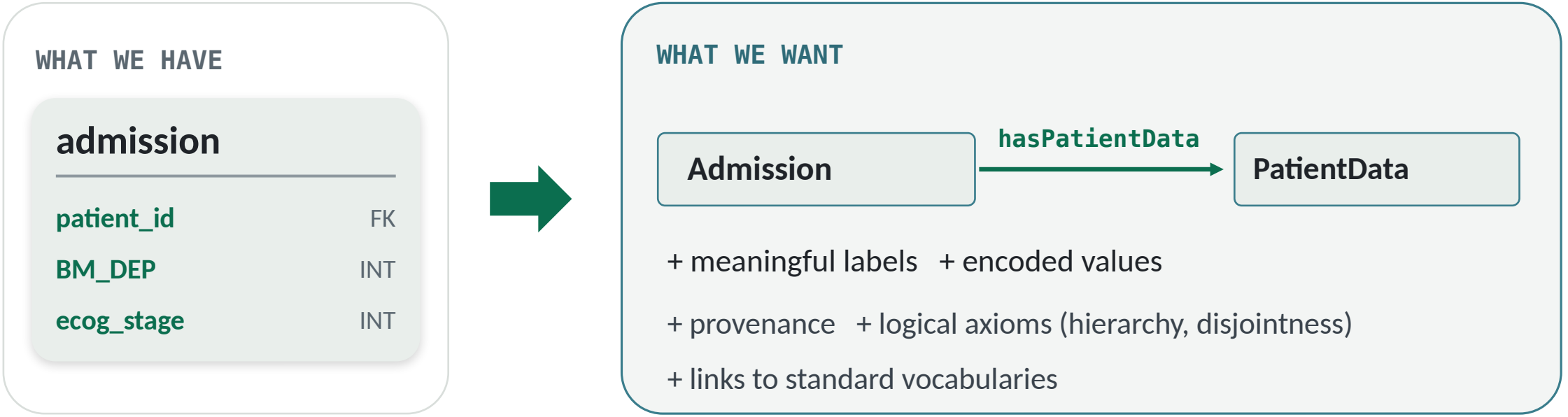
LLMs4OL [Babaei Giglou et al., ISWC 2023] · **Mateiu & Groza** [SYNASC 2023] · **OntoGenix** [Val-Calvo et al., IP&M 2025] · **LLM4VKG** [Xiao et al., IJCAI 2025]

OUR QUESTION **Can we preserve schema fidelity and add domain semantics across the full database?**

OUR ANSWER / RIGOR

Build the ontology one table at a time.

We will construct the answer to our admission example, together.



We need to preserve structure, discover meaning, and prevent mistakes, at database scale.

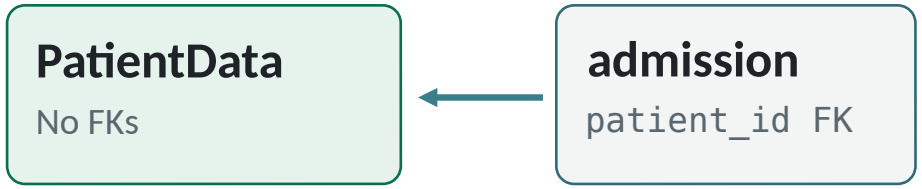
1 ORDER | 2 SEED | 3 RETRIEVE | 4 GENERATE | 5 VALIDATE | 6 MERGE

STEP 1 / ORDER

To understand this table, build its parent first.

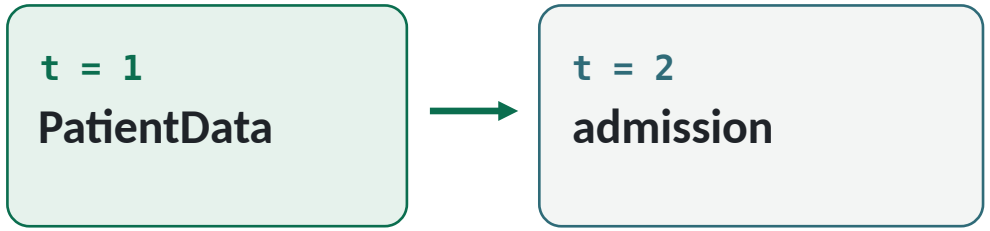
We follow foreign-key dependencies, so known concepts are available when needed.

DATABASE DEPENDENCIES



FK points to parent

PROCESSING ORDER / OUTPUT



Before generating `admission`, `PatientData` already exists in the core ontology.

WHY IT MATTERS The next fragment can **reuse** known **classes** and **IRIs** instead of inventing duplicates.

STEP 2 / SEED

Guarantee schema coverage before LLM enrichment.

Direct Mapping (DM_r) represents every column in the table before generation.

INPUT

admission

patient_id	FK → PatientData
BM_DEP	INT
ecog_stage	INT

OUTPUT / DM_r

```
Class: Admission
  SubClassOf:
    admission_id some xsd:integer,
    patient_id some PatientData,        ← each has a patient
    BM_DEP only xsd:integer,            ← if present, an integer
    ecog_stage only xsd:integer
DataProperty: admission_id
ObjectProperty: patient_id              ← the FK becomes a link
DataProperty: BM_DEP
DataProperty: ecog_stage
```

GUARANTEE Every schema column is represented in the deterministic seed.

1 ORDER | 2 SEED | 3 RETRIEVE | 4 GENERATE | 5 VALIDATE | 6 MERGE

STEP 3 / RETRIEVE

The draft is structurally faithful. But what does it mean?

RIGOR gathers a **context** package from **three** complementary **sources**.

1 DOCUMENTATION



What do column names mean?

`BM_DEP` → Bone Marrow Depression

Defines cryptic names

STEP 3 / RETRIEVE

The draft is structurally faithful. But what does it mean?

RIGOR gathers a **context** package from **three** complementary **sources**.

1 DOCUMENTATION

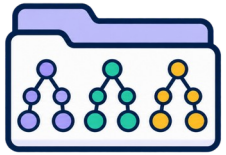


What do column names mean?

BM_DEP → Bone Marrow Depression

Defines cryptic names

2 EXTERNAL ONTOLOGIES



Which domain concepts exist?

Disease Ontology
Cell Ontology

Suggests established concepts

STEP 3 / RETRIEVE

The draft is structurally faithful. But what does it mean?

RIGOR gathers a **context** package from **three** complementary **sources**.

1 DOCUMENTATION

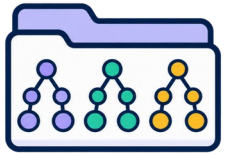


What do column names mean?

BM_DEP → Bone Marrow Depression

Defines cryptic names

2 EXTERNAL ONTOLOGIES

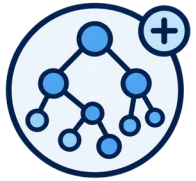


Which domain concepts exist?

Disease Ontology
Cell Ontology

Suggests established concepts

3 GROWING CORE



What have we already defined?

PatientData + previously accepted ontology terms

Reuses established concepts

1 ORDER | 2 SEED | **3 RETRIEVE** | 4 GENERATE | 5 VALIDATE | 6 MERGE

STEP 3 / RETRIEVE

The draft is structurally faithful. But what does it mean?

RIGOR gathers a **context** package from **three** complementary **sources**.

1 DOCUMENTATION

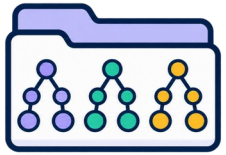


What do column names mean?

BM_DEP → Bone Marrow Depression

Defines cryptic names

2 EXTERNAL ONTOLOGIES

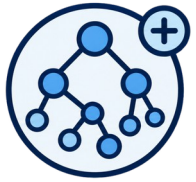


Which domain concepts exist?

Disease Ontology
Cell Ontology

Suggests established concepts

3 GROWING CORE



What have we already defined?

PatientData + previously accepted ontology terms

Reuses established concepts

ONE RETRIEVAL QUERY PER TABLE **table** + **columns** + **FK targets** → top-K per **source**

(e.g., ***admission → query: admission + patient_id, BM_DEP, ecog_stage + PatientData****)*

1 ORDER | 2 SEED | **3 RETRIEVE** | 4 GENERATE | 5 VALIDATE | 6 MERGE

STEP 4 / GENERATE

The LLM enriches a schema-grounded draft.

We prompt the ***Gen-LLM*** with the **direct mapping**, the **retrieved context**, and **semantic enrichment instructions**.

BEFORE / DIRECT MAPPING

```
Class: Admission
  SubClassOf:
    admission_id some xsd:integer,
    patient_id some PatientData,
    BM_DEP only xsd:integer,
    ecog_stage only xsd:integer

DataProperty: admission_id
ObjectProperty: patient_id
DataProperty: BM_DEP



DataProperty: ecog_stage
```

```
Class: Admission
  SubClassOf:
    admission_id some xsd:integer,
    patient_id some PatientData,
    BM_DEP only xsd:integer,
    ecog_stage only xsd:integer

DataProperty: admission_id
ObjectProperty: patient_id
DataProperty: BM_DEP



DataProperty: ecog_stage
```

1 ORDER | 2 SEED | 3 RETRIEVE | **4 GENERATE** | 5 VALIDATE | 6 MERGE

STEP 4 / GENERATE

The LLM enriches a schema-grounded draft.

We prompt the ***Gen-LLM*** with the **direct mapping**, the **retrieved context**, and **semantic enrichment instructions**.

BEFORE / DIRECT MAPPING

```
Class: Admission
  SubClassOf:
    admission_id some xsd:integer,
    patient_id some PatientData,
    BM_DEP only xsd:integer,
    ecog_stage only xsd:integer

DataProperty: admission_id
ObjectProperty: patient_id
DataProperty: BM_DEP



DataProperty: ecog_stage
```

Gen-LLM



AFTER / ENRICHED ΔO_r (delta ontology)

```
Class: Admission
  SubClassOf:
    admission_id some xsd:integer,
    hasPatientData some PatientData,                    SCHEMA
    BM_DEP only xsd:integer,
    ecog_stage only xsd:integer

DataProperty: admission_id
ObjectProperty: hasPatientData                          SCHEMA
DataProperty: BM_DEP
  Annotations:
    rdfs:label "Bone Marrow Depression"@en,             DOCS
    skos:relatedMatch obo:DOID_4961,                    ONTOLOGY
    prov:wasDerivedFrom admission:BM_DEP                SCHEMA
DataProperty: ecog_stage
  Annotations:
    rdfs:label "ECOG Performance Status"@en,            DOCS
    rdfs:comment "0 = fully active … 5 = dead"@en,      LLM
    prov:wasDerivedFrom admission:ecog_stage            SCHEMA
```

1 ORDER | 2 SEED | 3 RETRIEVE | **4 GENERATE** | 5 VALIDATE | 6 MERGE

STEP 5 / VALIDATE

Why validate? Plausible fragments can still break the ontology.

STRUCTURAL ERROR · GRAPH CHECKS



The LLM links **BM_DEP to a disease**:

```
ObjectProperty: BM_DEP  Range: BoneMarrowDepression
```

But the seed already has **`DataProperty: BM_DEP`**, a number. One property can't be both a number and a link: OWL 2 DL forbids it. Graph checks **reject it**.

1 ORDER | 2 SEED | 3 RETRIEVE | 4 GENERATE | 5 VALIDATE | 6 MERGE

STEP 5 / VALIDATE

Why validate? Plausible fragments can still break the ontology.

STRUCTURAL ERROR · GRAPH CHECKS



The LLM links **BM_DEP to a disease**:

```
ObjectProperty: BM_DEP  Range: BoneMarrowDepression
```

But the seed already has **DataProperty: BM_DEP**, a number. One property can't be both a number and a link: OWL 2 DL forbids it. Graph checks **reject it**.

SEMANTIC ERROR · JUDGE-LLM



Generated link for **admission**:

```
ObjectProperty: hasPatient  Range: Patient
```

A new class, but the **core ontology** already has **PatientData**: **two names** for the **same thing**. Valid OWL, so no rule complains. The **JUDGE-LLM** sees the core and asks to **reuse** it.

STEP 5 / VALIDATE

Why validate? Plausible fragments can still break the ontology.

STRUCTURAL ERROR · GRAPH CHECKS



The LLM links **BM_DEP to a disease**:

```
ObjectProperty: BM_DEP  Range: BoneMarrowDepression
```

But the seed already has `DataProperty: BM_DEP`, a number. One property can't be both a number and a link: OWL 2 DL forbids it. Graph checks **reject it**.

SEMANTIC ERROR · JUDGE-LLM



Generated link for `admission`:

```
ObjectProperty: hasPatient  Range: Patient
```

A new class, but the **core ontology** already has `PatientData`: **two names** for the **same thing**. Valid OWL, so no rule complains. The `JUDGE-LLM` sees the core and asks to **reuse** it.

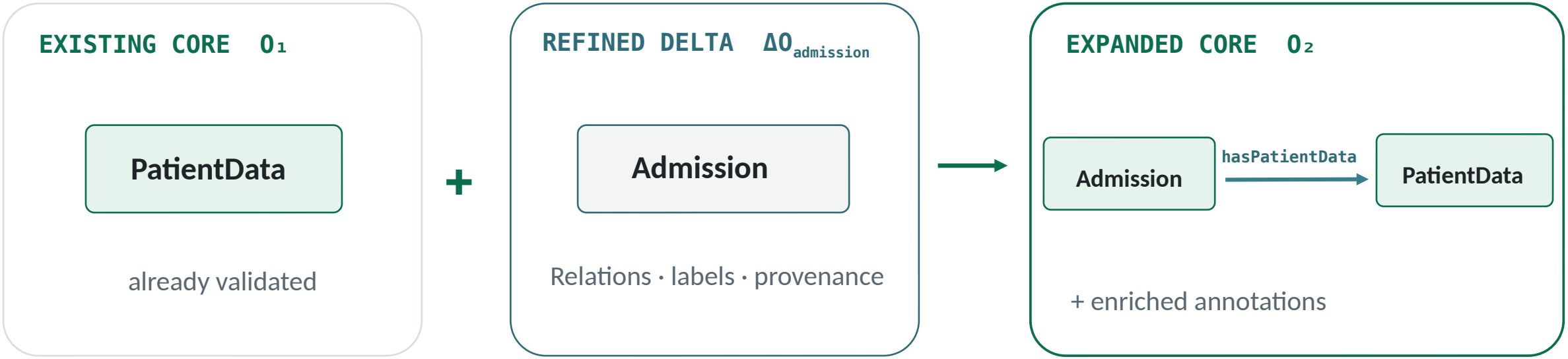
Problems go back to the ***Gen-LLM*** (up to 2 retries). If it **still fails**, the ***Judge-LLM*** picks the better of the ***Gen-LLM***'s final version and the plain ***direct mapping***.

1 ORDER | 2 SEED | 3 RETRIEVE | 4 GENERATE | **5 VALIDATE** | 6 MERGE

STEP 6 / MERGE

The ontology we built now helps build the next part.

Integrate the refined fragment; use the **expanded** core as retrieval context.

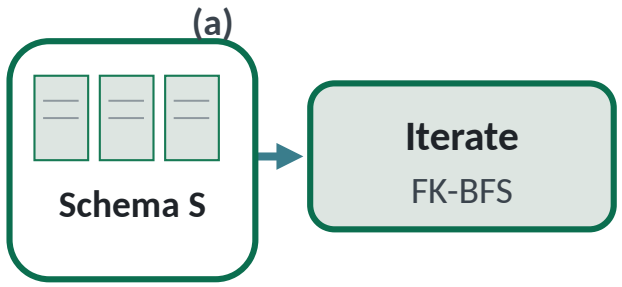


THE LOOP O_2 becomes a retrieval source for the next table → a coherent ontology **grows**.

1 ORDER | 2 SEED | 3 RETRIEVE | 4 GENERATE | 5 VALIDATE | 6 MERGE

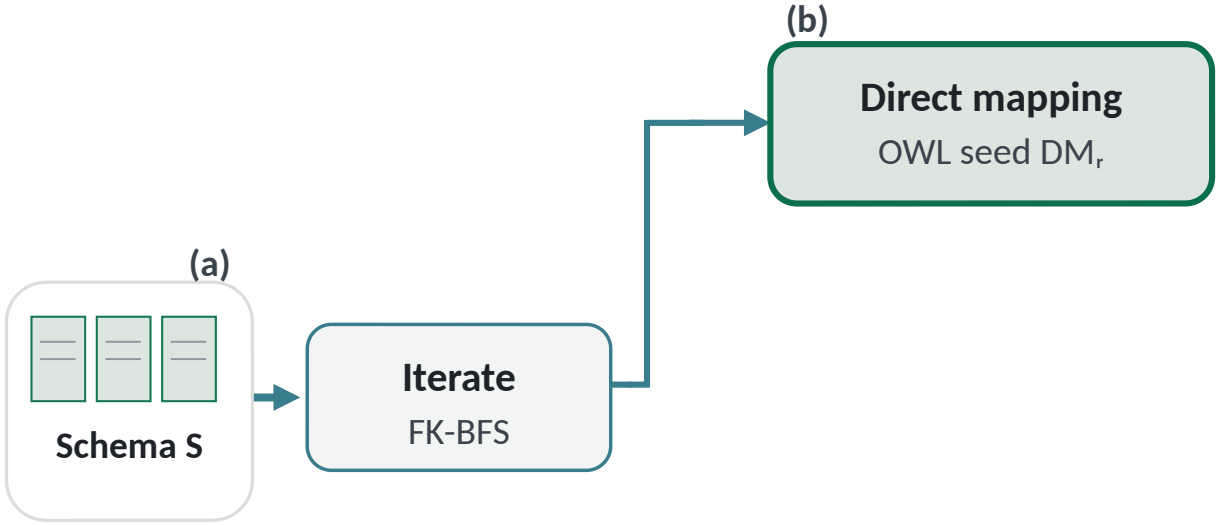
1. Follow the foreign keys

Start where the foreign-key dependencies begin.



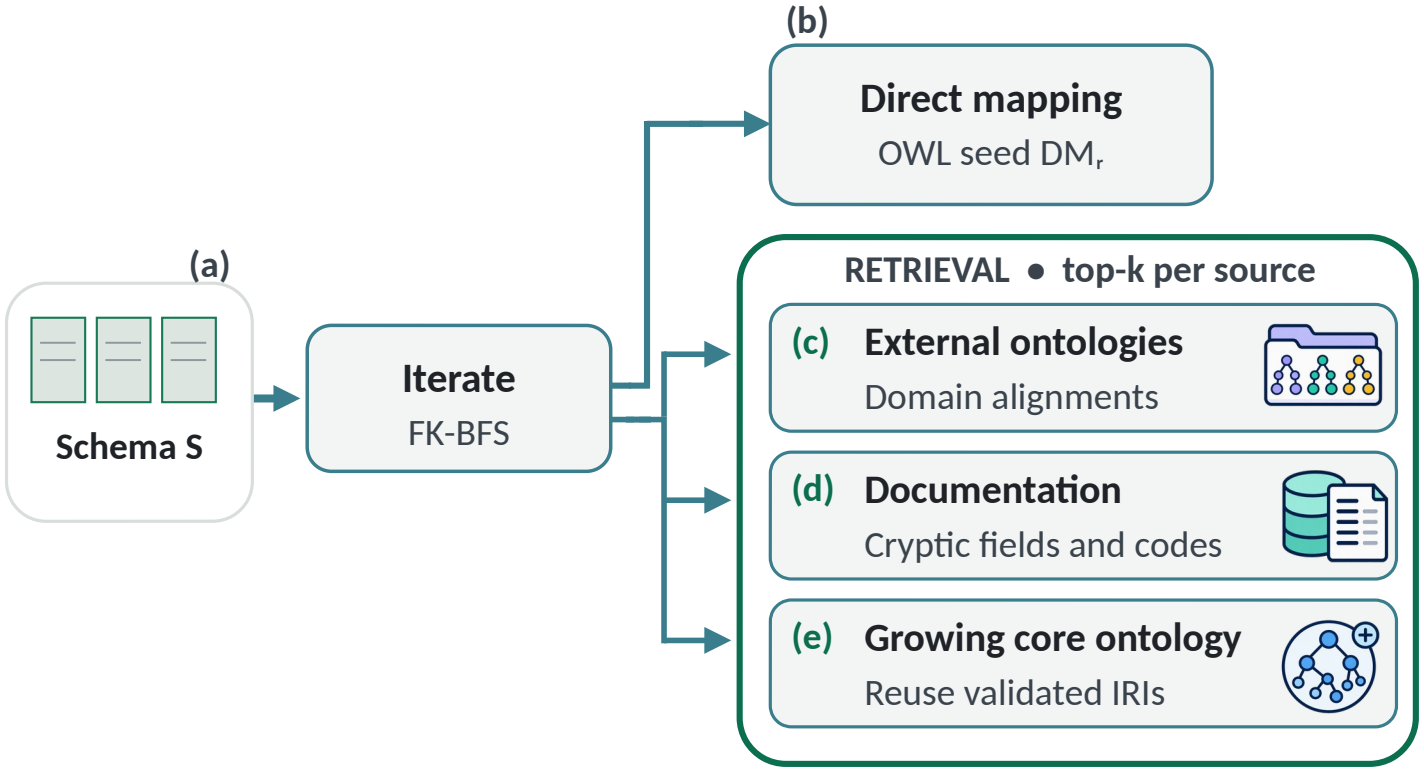
2. Anchor generation in the schema

The schema-grounded branch feeds the generation step.



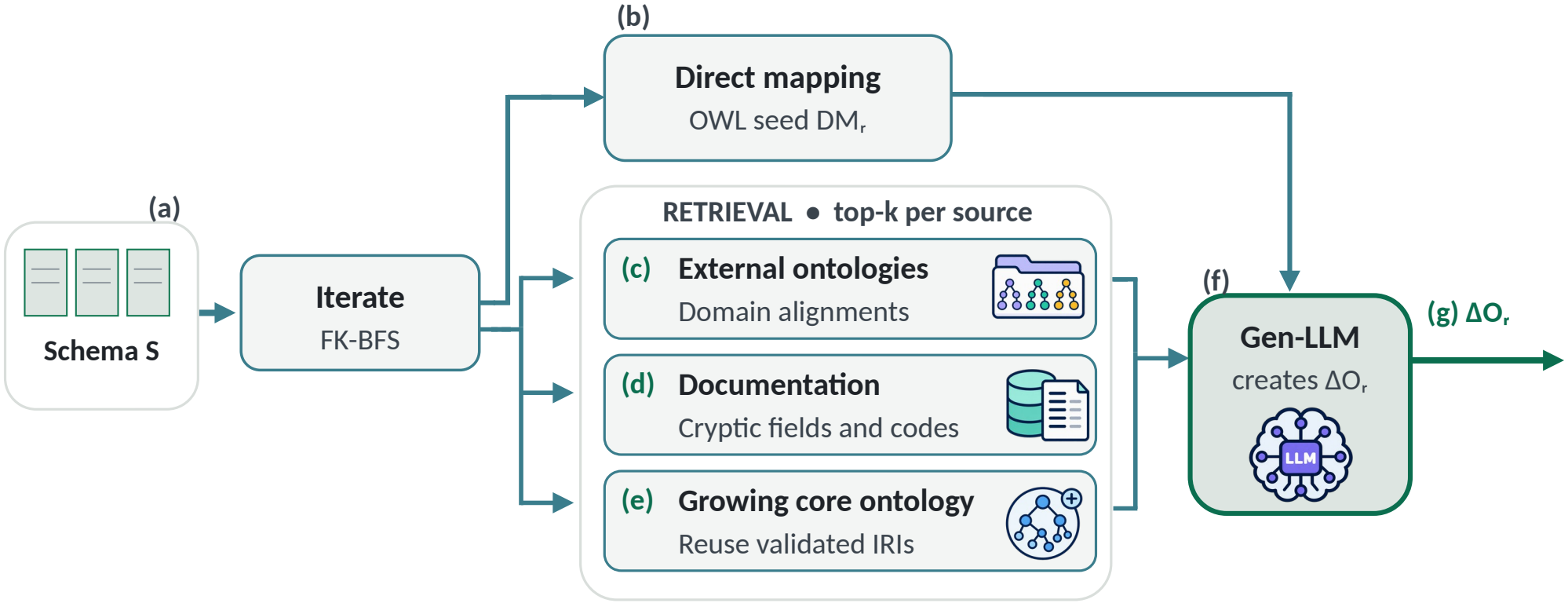
3. Retrieve the missing semantics

Context is retrieved independently from three sources.



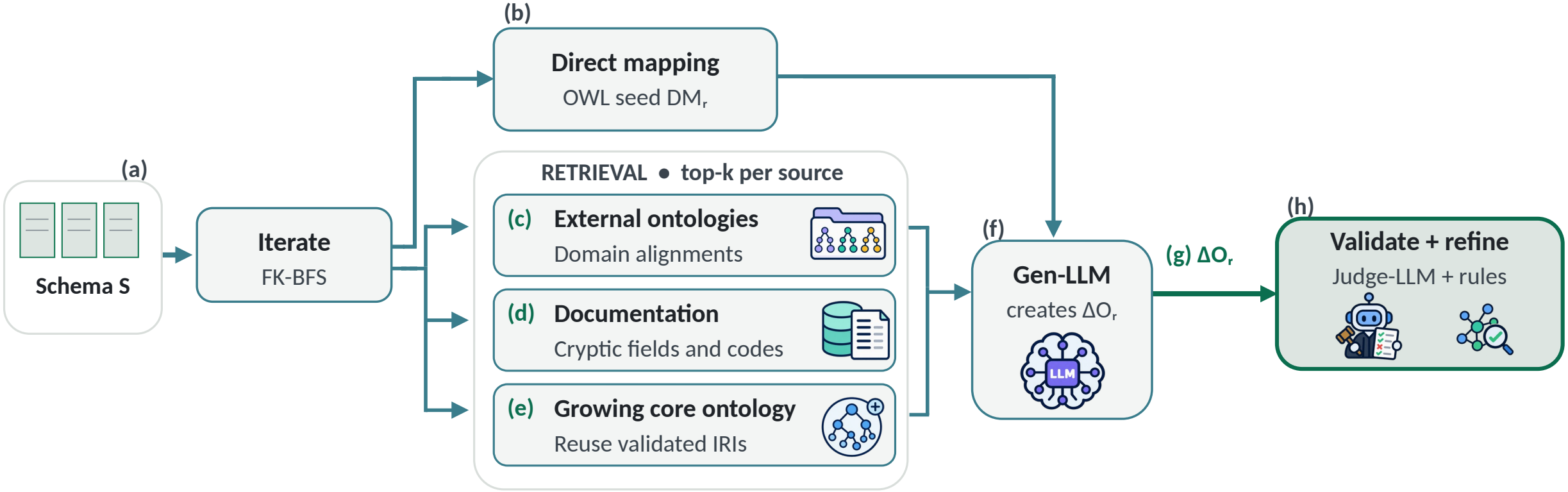
4. Generate a delta ontology

The Gen-LLM receives the direct mapping and the retrieved context.



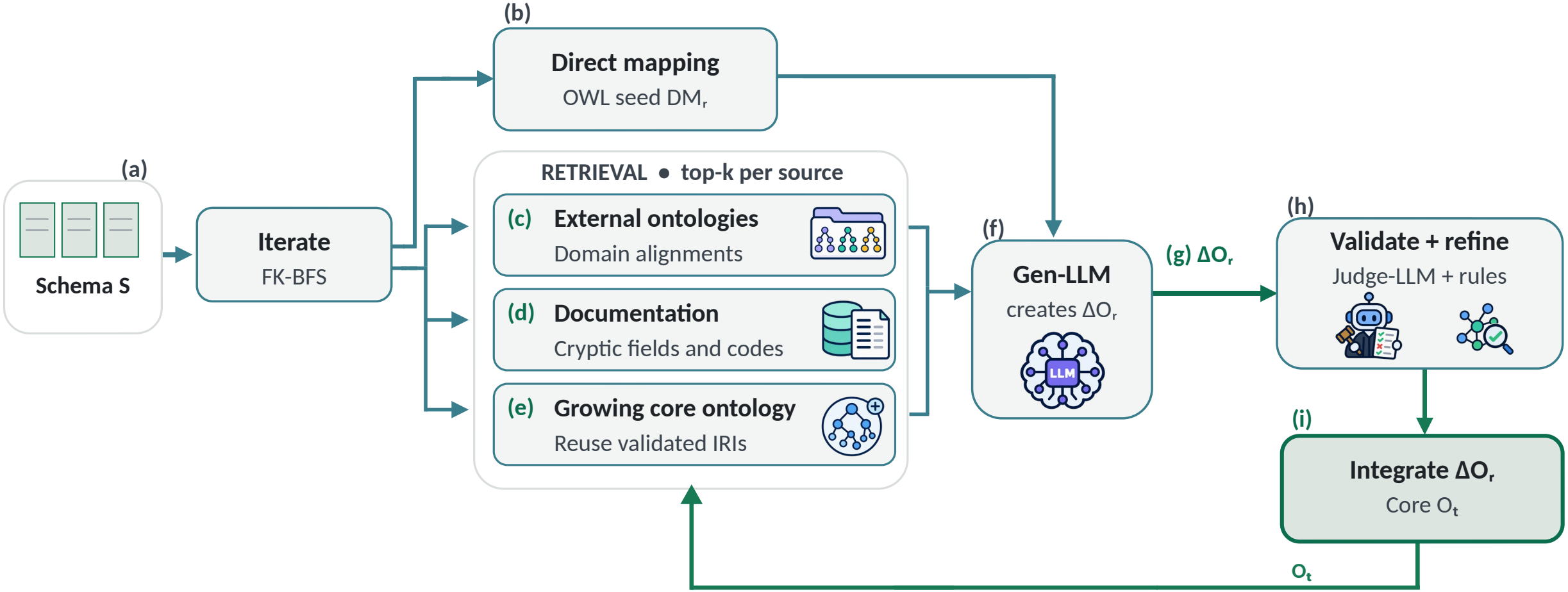
5. Validate before integration

Validate and refine each delta before it joins the core.



6. Integrate and reuse the growing core

The growing core feeds the next generation iteration.



EVALUATION

Does the method work beyond our running example?

Three databases, **three** generator models, **four** methods, evaluated from complementary **angles**.

LIVER CANCER



16 tables · 350 columns

specialist clinical registry

Universitätsklinikum Leipzig

eICU-CRD



31 tables · 559 columns

large ICU schema

PhysioNet [Pollard et al., Sci. Data 2018]

Medical Domain

EVALUATION

Does the method work beyond our running example?

Three databases, **three** generator models, **four** methods, evaluated from complementary **angles**.

LIVER CANCER



16 tables · 350 columns

specialist clinical registry

Universitätsklinikum Leipzig

eICU-CRD



31 tables · 559 columns

large ICU schema

PhysioNet [Pollard et al., Sci. Data 2018]

Medical Domain

CHINOOK



11 tables · 67 columns

music store

Chinook Database [Rocha n.d.; accessed 2026]

Music Domain

EVALUATION

Does the method work beyond our running example?

Three databases, **three** generator models, **four** methods, evaluated from complementary **angles**.

LIVER CANCER



16 tables · 350 columns

specialist clinical registry

Universitätsklinikum Leipzig

eICU-CRD



31 tables · 559 columns

large ICU schema

PhysioNet [Pollard et al., Sci. Data 2018]

CHINOOK



11 tables · 67 columns

music store

Chinook Database [Rocha n.d.; accessed 2026]

FOUR METHODS TO GENERATE AN ONTOLOGY FROM A RELATIONAL DATABASE SCHEMA

1 Direct Mapping

Deterministic

Direct Mapping [Sequeda et al., WWW 2012]

2 LLM baseline

Table only

Ontology engineering with large language models [Mateiu & Groza, SYNASC 2023]

3 Non-Iterative RAG

One-pass RAG

4 RIGOR

Ground + grow + check

LLM-based Methods

EVALUATION

Does the method work beyond our running example?

Three databases, **three** generator models, **four** methods, evaluated from complementary **angles**.

LIVER CANCER



16 tables · 350 columns

specialist clinical registry

Universitätsklinikum Leipzig

eICU-CRD



31 tables · 559 columns

large ICU schema

PhysioNet [Pollard et al., Sci. Data 2018]

CHINOOK



11 tables · 67 columns

music store

Chinook Database [Rocha n.d.; accessed 2026]

FOUR METHODS TO GENERATE AN ONTOLOGY FROM A RELATIONAL DATABASE SCHEMA

1 Direct Mapping

Deterministic

Direct Mapping [Sequeda et al., WWW 2012]

2 LLM baseline

Table only

Ontology engineering with large language models [Mateiu & Groza, SYNASC 2023]

3 Non-Iterative RAG

One-pass RAG

4 RIGOR

Ground + grow + check

LLM-based Methods

3 **LLM METHODS** × 3 **DATABASES** × 3 **LLMS** = **27 ONTOLOGIES**

EVALUATION

Does the method work beyond our running example?

Three databases, **three** generator models, **four** methods, evaluated from complementary **angles**.

LIVER CANCER



16 tables · 350 columns

specialist clinical registry

Universitätsklinikum Leipzig

eICU-CRD



31 tables · 559 columns

large ICU schema

PhysioNet [Pollard et al., Sci. Data 2018]

CHINOOK



11 tables · 67 columns

music store

Chinook Database [Rocha n.d.; accessed 2026]

FOUR METHODS TO GENERATE AN ONTOLOGY FROM A RELATIONAL DATABASE SCHEMA

1 Direct Mapping

Deterministic

Direct Mapping [Sequeda et al., WWW 2012]

2 LLM baseline

Table only

Ontology engineering with large language models [Mateiu & Groza, SYNASC 2023]

3 Non-Iterative RAG

One-pass RAG

4 RIGOR

Ground + grow + check

LLM-based Methods

1 **METHOD** × 3 **DATABASES** = 3 ONTOLOGIES

3 **LLM METHODS** × 3 **DATABASES** × 3 **LLMS** = 27 ONTOLOGIES

EVALUATION

Does the method work beyond our running example?

Three databases, **three** generator models, **four** methods, evaluated from complementary **angles**.

LIVER CANCER



16 tables · 350 columns

specialist clinical registry

Universitätsklinikum Leipzig

eICU-CRD



31 tables · 559 columns

large ICU schema

PhysioNet [Pollard et al., Sci. Data 2018]

CHINOOK



11 tables · 67 columns

music store

Chinook Database [Rocha n.d.; accessed 2026]

FOUR METHODS TO GENERATE AN ONTOLOGY FROM A RELATIONAL DATABASE SCHEMA

1 Direct Mapping

Deterministic

Direct Mapping [Sequeda et al., WWW 2012]

2 LLM baseline

Table only

Ontology engineering with large language models [Mateiu & Groza, SYNASC 2023]

3 Non-Iterative RAG

One-pass RAG

4 RIGOR

Ground + grow + check

LLM-based Methods

1 **METHOD** × 3 **DATABASES** = 3 ONTOLOGIES

3 **LLM METHODS** × 3 **DATABASES** × 3 **LLMS** = 27 ONTOLOGIES

3+27 = 30 ONTOLOGIES

EVALUATION

Does the method work beyond our running example?

Three databases, **three** generator models, **four** methods, evaluated from complementary **angles**.

LIVER CANCER



16 tables · 350 columns

specialist clinical registry

Universitätsklinikum Leipzig

eICU-CRD



31 tables · 559 columns

large ICU schema

PhysioNet [Pollard et al., Sci. Data 2018]

CHINOOK



11 tables · 67 columns

music store

Chinook Database [Rocha n.d.; accessed 2026]

FOUR METHODS TO GENERATE AN ONTOLOGY FROM A RELATIONAL DATABASE SCHEMA

1 Direct Mapping

Deterministic

Direct Mapping [Sequeda et al., WWW 2012]

2 LLM baseline

Table only

Ontology engineering with large language models [Mateiu & Groza, SYNASC 2023]

3 Non-Iterative RAG

One-pass RAG

4 RIGOR

Ground + grow + check

30 ONTOLOGIES → checked by quality checks · LLM-as-a-Judge (GPT-5.4) · 10 human experts · BURR benchmark [1]

[1] BURR benchmark [Laskowski et al., PACMMOD 2025]

RESULT 1 / VALID, WITH FEWER ERRORS

Every ontology is valid. RIGOR's have far fewer modeling errors.

All 30 ontologies **parse without errors** (RDFLib) and are **logically consistent** (HermiT). So we look deeper.

1 VALID, FEWER ERRORS | 2 KEEPS THE SCHEMA | 3 ADDS MEANING | 4 GOLD MAPPINGS | 5 JUDGES

MODELING ERRORS FOUND BY OOPS!

pitfalls per ontology, fewer is better

	Liver Cancer			eICU-CRD			Chinook			Avg
Direct Mapping	731			665			102			499
	Claude	Mistral	DeepSeek	Claude	Mistral	DeepSeek	Claude	Mistral	DeepSeek	
LLM baseline	422	397	403	434	440	421	95	74	81	307
Non-Iterative RAG	29	61	18	39	220	27	24	35	14	52
RIGOR	**20**	**23**	**3**	**2**	**2**	**26**	**0**	**0**	**0**	**8**

Same LLM, same database: **Mistral on eICU, 220 with Non-Iterative RAG**, **2 with RIGOR.** On Chinook, RIGOR has zero with every LLM.

OOPS! [Poveda-Villalón et al., IJSWIS 2014]

RESULT 2 / NOTHING IS LOST

Nothing is lost: RIGOR keeps the schema.

1 VALID, FEWER ERRORS | 2 KEEPS THE SCHEMA | 3 ADDS MEANING | 4 GOLD MAPPINGS | 5 JUDGES

KEEPS THE SCHEMA

columns covered, per ontology (%)

	Liver Cancer			eICU-CRD			Chinook			Avg
	Claude	Mistral	DeepSeek	Claude	Mistral	DeepSeek	Claude	Mistral	DeepSeek	
Direct Mapping	97			99			78			91%
LLM baseline	94	31	89	96	68	86	100	50	52	74%
Non-Iterative RAG	41	6	12	22	28	46	58	16	78	34%
RIGOR	**91**	**99**	**87**	**100**	**99**	**77**	**95**	**95**	**95**	**93%**

Coverage: a column counts as covered when an ontology class or property name is similar to it (embedding similarity ≥ 0.55).

RESULT 2 / NOTHING IS LOST

Nothing is lost: RIGOR keeps the schema.

1 VALID, FEWER ERRORS | **2 KEEPS THE SCHEMA** | 3 ADDS MEANING | 4 GOLD MAPPINGS | 5 JUDGES

KEEPS THE SCHEMA

columns covered, per ontology (%)

	Liver Cancer			eICU-CRD			Chinook			Avg
	Claude	Mistral	DeepSeek	Claude	Mistral	DeepSeek	Claude	Mistral	DeepSeek	
Direct Mapping	97			99			78			91%
LLM baseline	94	31	89	96	68	86	100	50	52	74%
Non-Iterative RAG	41	6	12	22	28	46	58	16	78	34%
RIGOR	**91**	**99**	**87**	**100**	**99**	**77**	**95**	**95**	**95**	**93%**

Non-Iterative RAG 6–78%. With RIGOR: **≥ 77% for every LLM, in every database.**

Coverage: a column counts as covered when an ontology class or property name is similar to it (embedding similarity ≥ 0.55).

RESULT 3 / MEANING IS ADDED

Meaning is added: labels, provenance and hierarchy.

1 VALID, FEWER ERRORS | 2 KEEPS THE SCHEMA | **3 ADDS MEANING** | 4 GOLD MAPPINGS | 5 JUDGES

ADDS MEANING

		Liver Cancer			eICU-CRD			Chinook			Avg
		Claude	Mistral	DeepSeek	Claude	Mistral	DeepSeek	Claude	Mistral	DeepSeek	
	Direct Mapping LLM baseline	*No labels, provenance or class hierarchy, in any run.*									
Labels & comments	Non-Iterative RAG	680	134	736	877	157	743	98	14	64	389
	RIGOR	**1,029**	**1,137**	**341**	**1,291**	**1,684**	**435**	**357**	**362**	**359**	**777**
Provenance links	Non-Iterative RAG	0	0	0	0	0	0	0	0	0	0
	RIGOR	**336**	**308**	**22**	**423**	**335**	**99**	**81**	**83**	**75**	**196**
Class hierarchy *(inheritance richness)*	Non-Iterative RAG	0	0	0	0	0	0	0	0	0	0
	RIGOR	**0.70**	**0.39**	**0.13**	**0.62**	**0.49**	**0.10**	**2.12**	**2.72**	**4.07**	**1.26**

Only RIGOR adds **labels**, **provenance and** a **class hierarchy**: **with every LLM, in every database.**

Labels & comments: annotation assertions; provenance: prov:wasDerivedFrom.
Hierarchy: inheritance richness, subclass axioms per class [**OntoQA:** Tartir et al., KADASH 2005; **OntoMetrics:** Lantow, KEOD 2016].

RESULT 4 / EXTERNAL BENCHMARK

RIGOR keeps the link to the data.

BURR [Laskowski et al., PACMMOD 2025]

1 VALID, FEWER ERRORS | 2 KEEPS THE SCHEMA | 3 ADDS MEANING | **4 GOLD MAPPINGS** | 5 JUDGES

WHAT BURR CHECKS

Every ontology element must map back to the database.

CLASSES	`person`	→	`Class: Person`
RELATIONS	`paper.author_id`	→	`ObjectProperty: author`
ATTRIBUTES	`person.name`	→	`DataProperty: name`

SYSTEM	CLASSES	RELATIONS	ATTRIBUTES
W3C-Mapper / Ontop rule-based	0.53	0.11	0.39
RDB2Onto rule-based	*0.67*	*0.62*	*0.44*
OntoGenix LLM-based	0.58	0.35	*0.44*
RIGOR LLM + rules	**0.73**	**0.67**	**0.52**

RESULT 5 / LLM JUDGE AND HUMAN EXPERTS

An LLM judge and human experts both rank RIGOR first.

Blind evaluation: nobody knew which ontology came from which method.

1 VALID, FEWER ERRORS | 2 KEEPS THE SCHEMA | 3 ADDS MEANING | 4 GOLD MAPPINGS | **5 JUDGES**

HOW THEY JUDGED For each table: its **schema**, **5 competency questions** and **four anonymized ontologies**. Each **scored 0–5** on ***accuracy***, ***completeness***, ***conciseness***, ***adaptability***, ***clarity***, ***consistency*** and ***domain enrichment***, then ranked **1st–4th**.

LLM-AS-A-JUDGE · GPT-5.4

100% of rankings put RIGOR 1st

	Liver Cancer	eICU-CRD
Direct Mapping	2.05	2.13
LLM baseline	1.38	0.28
Non-Iterative RAG	1.89	0.67
RIGOR	**3.93**	**3.64**

10 ONTOLOGY EXPERTS

70% of rankings put RIGOR 1st

	Liver Cancer	eICU-CRD
Direct Mapping	2.95	2.59
LLM baseline	2.74	3.01
Non-Iterative RAG	2.39	1.24
RIGOR	**3.44**	**3.53**

Quality dimensions: accuracy, completeness, conciseness, adaptability, clarity, consistency [Vrandečić, Handbook on Ontologies 2009]; domain enrichment, adapted from ontology enrichment [Petasis et al., LNCS 2011].

FUTURE WORK

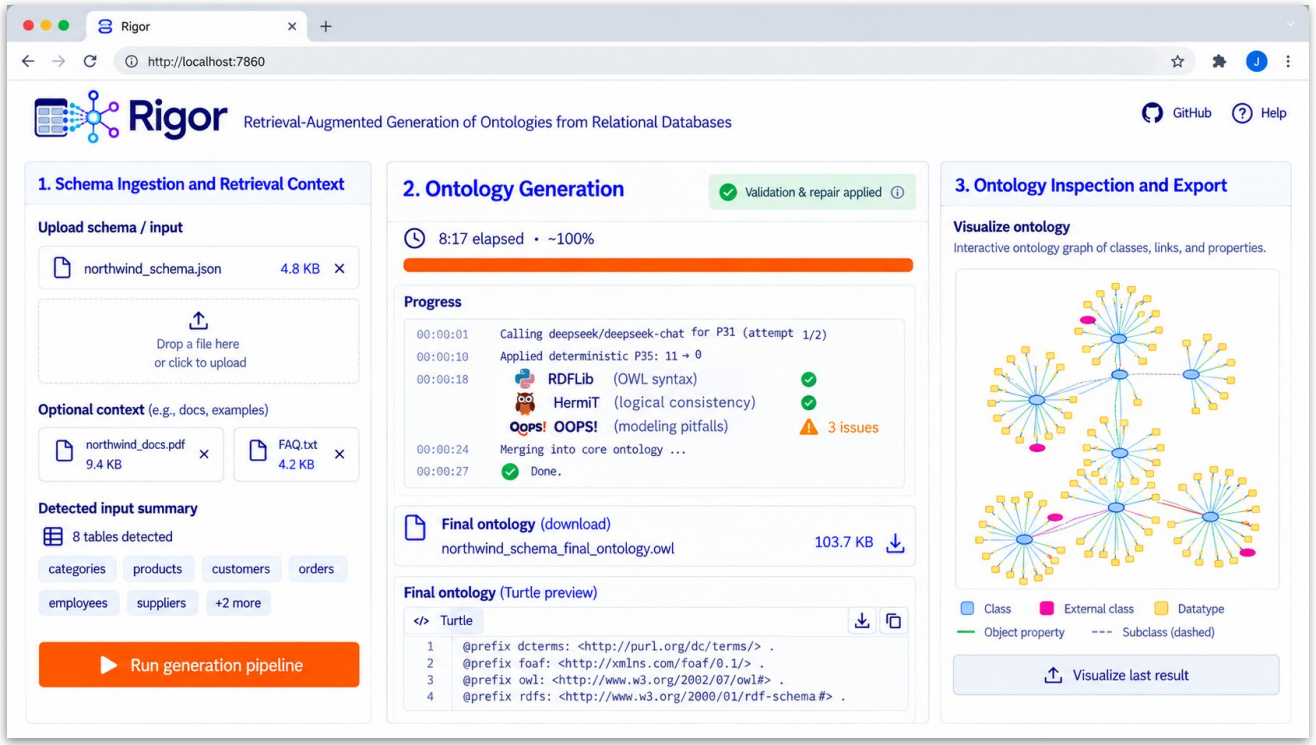
Next: Move the evaluation tools into the loop

RDFLib, the **reasoner** and **OOPS!** only checked our final ontologies. Next, they flag each error while ***RIGOR*** builds, and their **feedback** will be used to **prompt the LLM** to **repair** the ontology.

FUTURE WORK

Next: Move the evaluation tools in the loop

RDFLib, the **reasoner** and **OOPS!** only checked our final ontologies. Next, they flag each error while ***RIGOR*** builds, and their **feedback** will be used to **prompt the LLM** to **repair** the ontology. → *Live in our RIGOR App demo.*



RIGOR App [Fathallah, Nayyeri & Staab, ISWC 2026 Demos].

Open source · MIT

Watch the demo video

FUTURE WORK

Next: let the data speak.

Today ***RIGOR*** reads **schemas**. **Instance data** can propose **new knowledge**.

1 THE DATA

two tables, joined on admission_id

treatment

admission_id FK	drug
A1	cisplatin
A2	nivolumab
A3	cisplatin

admission

admission_id	patient_id	BM_DEP
A1	P1	1
A2	P2	0
A3	P3	1

FUTURE WORK

Next: let the data speak.

Today ***RIGOR*** reads **schemas**. **Instance data** can propose **new knowledge**.

1 THE DATA

two tables, joined on admission_id

treatment

admission_id FK	drug
A1	cisplatin
A2	nivolumab
A3	cisplatin

admission

admission_id	patient_id	BM_DEP
A1	P1	1
A2	P2	0
A3	P3	1

2 THE PATTERN

what the joined rows show

Every cisplatin admission has bone marrow depression.



admissions with BM_DEP = 1

3 CANDIDATE KNOWLEDGE

A reviewable hypothesis

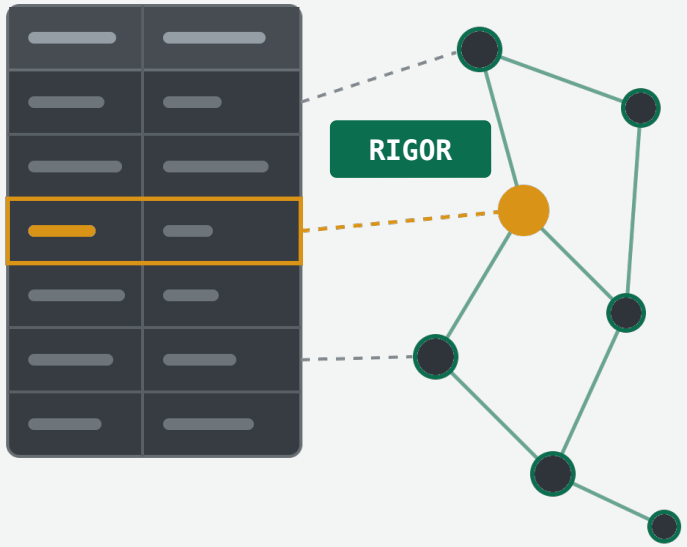
```
Individual: cisplatin_bmdep
  Types: CandidateAssociation
  Facts:
    hasDrug Cisplatin,
    hasCondition BoneMarrowDepression,
    status "needs expert review"
  Annotations:
    prov:wasDerivedFrom treatment:drug,
    prov:wasDerivedFrom admission:BM_DEP
```

A PATTERN → an expert reviews it before it joins the ontology

Thank you.

Questions?

FROM A DATABASE SCHEMA TO AN ONTOLOGY



PAPER + OPEN SOURCE

Paper

GitHub

📬 nadeen.fathallah@ki.uni-stuttgart.de

Questions we expect

Click a question to jump to its answer; every answer links back here.

EVALUATION

B1 Could the LLM judge be biased?
B2 How were the CQs used? Why not answer them in SPARQL?
B3 Who were the experts, and do they agree?
B4 Which model did what?

RESULTS

B5 Why is coverage below 100%?
B6 Aren't the OOPS! counts just missing labels?
B7 Did you run an ablation?
B8 Which part of RIGOR matters most?

METHOD

B9 Why breadth-first foreign-key order?
B10 What exactly does validation check?
B11 Why these settings?

PRACTICE

B12 What does it cost, and does it scale?
B13 BURR: setup, and why not compare with Gemini?
B14 What are the limitations?

Could the LLM judge be biased?

Possibly, which is why it is never our only evidence: four independent checks point the same way.

BIAS	WHAT WE DID	WHAT REMAINS
Self-preference	Judge is GPT-5.4, a different model family from all three generators	Inside RIGOR, the Judge-LLM is the generator's own model (graph checks are model-free)
Position	Fragments anonymized as A–D	**Order was fixed (RIGOR always D), for judge and experts: re-judge with shuffled order**
Verbosity	Scored per dimension, not only ranked: the judge marks RIGOR down on conciseness where it is long (Chinook 2.59 vs 3.32)	Clarity and domain enrichment reward annotations by design
Leniency	Ten human experts, blind, same rubric: RIGOR 1st in 70%, significant agreement	Judge is harsher than experts on the LLM baseline (eICU 0.28 vs 3.01)

No LLM involved: **OOPS! pitfalls 8 vs 52–499, highest BURR F1 on all three.** Both agree with the judges.

Paper Sec. 4.4, 4.6, Tables 1, 7, 10–13; fragment order: Fig. 7 and expert_survey/METHOD_KEY.json.

↩ All questions

How were the CQs used? Why not answer them in SPARQL?

As a shared yardstick for blind judging: what each table's ontology must be able to answer.

HOW WE USED THEM

- **5 per table, 290 in total**, generated from the schema (Mistral Small 24B), verified by a domain expert
- **Never shown to the generator**, so they stay an independent test
- **Given to judge and experts** with the schema and the four anonymized fragments: can this ontology answer them?

Example (liver, general_aftercare)
Which patients developed bile-duct stenosis after surgery, and what treatment was documented?

WHY NOT RUN THEM AS SPARQL (YET)

- **No instance data:** the ontologies are schemas (TBox), so data questions return empty for every method
- **It would test the wrong thing:** answerability mostly checks that columns are kept (Result 2), not labels, value meanings or hierarchy
- **A second error source:** 4 × 290 hand- or LLM-written queries

Next: populate a KG through RIGOR's provenance mapping (already exported for BURR) and run the CQs as SPARQL.

Paper Sec. 4.2, 4.6 (Strategies 6–7), App. Table 6; cqs/ in the RIGOR repository.

↩ All questions

Who were the experts, and do they agree?

Ten ontology engineers, blind, on four tables; agreement is significant on every table.

TABLE	DATABASE	WHY THIS TABLE	KENDALL'S W	P
general_aftercare	Liver	clinical follow-up, several foreign keys	0.59	< 0.001
complication	Liver	encoded severity grades, part-whole	0.39	< 0.01
hospital	eICU	compact administrative entity	0.59	< 0.001
patient	eICU	hub referenced by most tables	0.72	< 0.001

RIGOR 1ST

14 / 20
liver

14 / 20
eICU

- **Who:** ten experts in ontology engineering and knowledge representation (profiles in paper Fig. 8)
- **Task:** per table, schema + 5 CQs + four anonymized fragments; score 7 dimensions 0–5, rank 1st–4th. Biggest RIGOR gains: clarity and domain enrichment
- **Why 4 tables, Claude only:** survey length (4 fragments × 7 scores + a ranking per table); the LLM judge covered all 58 tables

Paper Sec. 4.6 (Strategy 7), Tables 1, 11, 12, Fig. 8.

↩ All questions

BACKUP B4 / EVALUATION

Which model did what?

Generation, documentation, questions and judging are separated; no model grades its own work.

ROLE	MODEL	SAFEGUARD
Ontology generation	Claude Opus 4.6 · Mistral Small 24B · DeepSeek-V3	same models for all three LLM methods
Judge-LLM inside RIGOR	the generator's own model	plus deterministic graph checks
Documentation	GPT-4o	reviewed by domain experts
Competency questions	Mistral Small 24B	verified by a domain expert
Evaluation judge	GPT-5.4	different family from every generator
External ontologies	ICD-10, Disease, Cell, Nutrition · Music, Performed Music, DOREMUS	chosen by domain experts

Data leakage: the liver-cancer registry is private, so no model has seen it, and the judge still ranked RIGOR first on all 16 of its tables.

Paper Sec. 4.1, 4.3, 4.4, Tables 1, 4.

↩ All questions

Why is coverage below 100%?

Two different reasons: a name-based metric for Direct Mapping, real omissions in one RIGOR run.

DIRECT MAPPING ON CHINOOK: 78%

- **43 ordinary columns:** a property with exactly the same name, so all are matched
- **21 key columns:** primary keys become HasKey axioms, not properties; foreign keys are renamed (ArtistId → hasArtist, ReportsTo → hasEmployee)
- **The metric compares names,** so most key columns fall below 0.55. Nothing was lost.

RIGOR WITH DEEPSEEK ON eICU: 77%

- **Real omissions:** the final ontology has 197 data properties; the seed had 299
- **Why:** in the paper's pipeline only the validated delta is merged, so a delta that renames or drops seed properties loses columns
- **Fixed:** the current RIGOR code merges the direct mapping into the core first, so every column stays

Coverage: a column counts as covered when some ontology class or property name has embedding similarity ≥ 0.55 (all-MiniLM-L6-v2).

Paper Tables 8, 9, Algorithm 1 (line 22); output/generated_ontologies/direct_mapping_chinook.owl, app.py.

↩ All questions

Aren't the OOPS! counts just missing labels?

No: without P08, RIGOR still has the fewest pitfalls, and the fewest critical ones.

AVERAGE PER ONTOLOGY	ALL PITFALLS	WITHOUT P08	CRITICAL (P19)
Direct Mapping	499	250	15
LLM baseline	307	34	17
Non-Iterative RAG	52	29	5.4
RIGOR	**8**	**8**	**2.8**

- **P08 = missing annotations:** 72% of the other methods' pitfalls. Without it, RIGOR still has the fewest, about 4 times fewer than the next method
- **P19 = multiple domains or ranges,** the only critical pitfall found: zero in 6 of RIGOR's 9 ontologies

Paper Table 7, averaged over 3 (Direct Mapping) or 9 ontologies; global pitfalls without counts (P10, P22) excluded.

↩ All questions

Did you run an ablation?

Yes, for the retrieval sources: all sources together align best, but the margins are small.

VARIANT (CHINOOK, CLAUDE)	SCHEMA	EXT. ONTOLOGIES	DOCUMENTATION	AVERAGE
No retrieval	1.000	0.033	0.919	0.651
Schema + core only	0.997	0.033	0.916	0.648
External ontologies only	1.000	0.033	0.919	0.651
Documentation only	1.000	0.035	**0.921**	0.652
Full RIGOR	0.993	**0.059**	0.914	**0.655**

- **Full RIGOR aligns best overall** and nearly doubles external-ontology alignment; each source helps its own resource
- **Why the margins are small:** every variant keeps the direct-mapping seed, so schema alignment is saturated, and Chinook's names are self-explanatory

Paper Sec. 4.8, Tables 5, 14. F1 of matches (cosine ≥ 0.55) between ontology elements and each resource.

↩ All questions

Which part of RIGOR matters most?

Our four methods add one step at a time, so each step's effect is visible.

ADDS ONE STEP EACH	DIRECT MAPPING	+ LLM	+ RETRIEVAL	+ SEED, CORE, CHECKS
	rules only	LLM baseline	Non-Iterative RAG	**RIGOR**
Columns covered	91%	74%	**34%**	**93%**
OOPS! pitfalls	499	307	52	**8**
Labels & comments	0	0	389	**777**
Provenance links	0	0	0	**196**
Judge: ranked 1st	0 / 58	0 / 58	0 / 58	**58 / 58**

Reading: retrieval adds labels but drops columns; the seed brings the columns back; the checks cut the errors.

Not isolated: seed, growing core and validation were added together. A leave-one-component-out ablation is future work.

Paper Sec. 4.5 (controlled spectrum, App. Table 3), Tables 1, 7, 8, 9; averages per ontology.

↩ All questions

BACKUP B9 / METHOD

Why breadth-first foreign-key order?

Parents before children, so every table can reuse the classes it points to.

CHINOOK, IN PROCESSING ORDER

LEVEL 0	LEVEL 1	LEVEL 2	LEVEL 3
Artist	Album	Track	InvoiceLine
Employee	Customer	Invoice	PlaylistTrack
Genre			
MediaType			
Playlist			

Track waits for Album, Genre and MediaType.

WHY

- **Reuse:** the classes a table points to already exist, so the LLM links to their IRIs instead of inventing new ones
- **Context:** the parents' labels and comments are in the growing core, ready to retrieve
- **Reproducible:** level by level, ties broken alphabetically
- **Cycles:** a self-reference (Employee.ReportsTo) does not block the order and becomes a recursive relation; tables in a longer cycle are appended at the end

Any parents-first order works; we chose breadth-first for level-by-level growth. We did not compare orders.

Paper Sec. 3.1–3.2 (FK-BFS); app.py traverse_schema_fk_order; schema_chinook.json.

↩ All questions

What exactly does validation check?

Fourteen Judge-LLM criteria in three severities, plus four deterministic graph checks.

JUDGE-LLM: 14 CRITERIA

- **Critical:** FK never a data property; no URI both object and data property; exactly one domain and range; valid XSD ranges; no self-loops unless recursive; keep the direct mapping's coverage and key restrictions
- **Important:** English labels and comments; meanings of coded values; junction tables as relations; reuse core concepts; disjointness only when safe
- **Minor:** external alignments; provenance on every element

GRAPH CHECKS (RULES)

- Type consistency
- One domain and one range per property
- XSD-only data ranges
- Full schema coverage

On failure: issues go back to the Gen-LLM (k = 2 retries); then the Judge-LLM keeps the better of its last version and the direct mapping.

Paper Sec. 3.5, App. A.2 (Fig. 3), Algorithm 1.

Why these settings?

Each setting trades context against noise; values follow the literature or a small tuning run.

CHOICE	VALUE	WHY
Seed	W3C Direct Mapping	every column is in the ontology before enrichment [Sequeda et al., WWW 2012]
Order	foreign keys, breadth-first	parents before children (backup B9)
Embeddings	all-MiniLM-L6-v2	strong on MTEB, light enough to re-index every iteration
Retrieved items	k = 3 per source	enough context without crowding out the seed and instructions
Match threshold	cosine ≥ 0.55	empirically tuned, as in prior work [Galli et al. 2024; Fathallah et al. 2024]
Retries	k = 2	self-refinement gains fade after two rounds [Madaan et al. 2023; Shinn et al. 2023]
Syntax	OWL 2 Manchester	readable for experts, generated reliably by LLMs

Paper Sec. 3.2–3.5, 4.6.

↩ All questions

BACKUP B12 / PRACTICE

What does it cost, and does it scale?

All nine RIGOR runs: about $34 and 11 hours; the small open model costs cents.

DATABASE	CLAUDE OPUS 4.6	MISTRAL SMALL 24B	DEEPSEEK-V3
eICU-CRD · 31 tables	$21.30 · 182 min	$0.08 · 188 min	$0.18 · 21 min
Liver Cancer · 16	$10.30 · 94 min	$0.02 · 21 min	$0.07 · 24 min
Chinook · 11	$1.28 · 9 min	$0.02 · 65 min	$0.03 · 12 min

All nine runs: 651 requests · 5.3 M tokens · $34.28 · 11.4 hours

- **Scales with the number of tables:** one prompt per table, and k = 3 items per source keep each prompt bounded as the core grows
- **Privacy:** only the schema and documentation are sent, never patient rows; Mistral Small 24B can run on-premises and reaches 95–99% coverage with RIGOR

Paper Table 15 (runtime includes Gen-LLM and Judge-LLM calls), Table 9.

BURR: setup, and why not compare with Gemini?

ISWC scenario, mappings recovered from RIGOR's provenance; Gemini is a mapping generator.

SETUP

- ISWC scenario: 9 tables, 46 columns, 11 foreign keys
- RIGOR with Claude; documentation by GPT-5.4; SWRC and BIBO as external ontologies
- **Mappings come from provenance:** prov:wasDerivedFrom exported as a D2RQ mapping
- F1 against the gold mapping, per element type

SYSTEM	CLASSES	RELATIONS	ATTRIBUTES
W3C-Mapper / Ontop	0.53	0.11	0.39
RDB2Onto	0.67	0.62	0.44
OntoGenix	0.58	0.35	0.44
RIGOR	**0.73**	**0.67**	**0.52**
Gemini 2.5 Pro*	0.65	0.68	0.56

** In BURR, a general LLM prompted with examples to write the mapping directly: a mapping generator, not an ontology-learning system. RIGOR also builds labels, hierarchy and provenance.*

Paper Sec. 4.6 (Strategy 8), Table 13; Gemini: BURR [Laskowski et al., PACMMOD 2025], Table 3.

↩ All questions

What are the limitations?

What we know is missing, and what we would do about it.

COVERAGE 0.77–1.00

A delta can drop seed properties.

→ Merge the direct mapping first (RIGOR App).

RESIDUAL P19

Critical pitfall in 3 of 9 RIGOR runs.

→ Validators inside the loop (RIGOR App).

DISJOINTNESS, ALIGNMENTS

13 disjointness axioms in one run; external F1 ≤ 0.06.

→ Couple with a reasoner and entity linking.

EVALUATION DESIGN

Fixed fragment order; experts on 4 tables, Claude only.

→ Shuffled re-judging; more tables and models.

ABLATION

One database, one LLM; seed, core and checks not isolated.

→ Leave-one-component-out on all databases.

SCHEMA ONLY

No instance data, so CQs are judged, not executed.

→ Let the data speak (future work).

Paper Sec. 4.7–5, Tables 7–9, 14.